

Table des Matieres

1. Resume Executif	3
2. Architecture Globale - 7/10	4
2.1 Structure du projet	4
2.2 Points forts	4
2.3 Faiblesses identifiees	5
3. Securite - 5/10	5
3.1 Vulnerabilite critique : contournement du paiement	5
3.2 Vulnerabilites de niveau moyen	6
3.3 Points forts de securite	6
4. Experience Utilisateur - 8/10	6
4.1 Points forts UX	6
4.2 Lacunes identifiees	7
5. Fonctionnalites - 8/10	7
5.1 Cycle de vie des commandes	7
5.2 Fonctionnalites implementees	8
5.3 Fonctionnalites manquantes	8
5.4 Paiements simules	9
6. Qualite du Code - 6/10	9
6.1 Typage TypeScript	9
6.2 Validation des donnees	9
6.3 Gestion des erreurs	9
6.4 Duplication de code	10
7. Performance - 6/10	10
7.1 Chargement massif de donnees	10
7.2 Polling vs WebSocket	10
7.3 Absence de pagination cote serveur	10
7.4 Optimisations absentes	11
8. Maintenabilite - 5/10	11

8.1 Absence de tests automatisés	11
8.2 Composants monolithiques	12
8.3 Relations en chaîne de caractères	12
8.4 Absence de journal d'audit	12
9. Feuille de Route vers la Production	12
9.1 Phase 1 : Sécurité critique (2 semaines)	12
9.2 Phase 2 : Performance et stabilité (2 semaines)	13
9.3 Phase 3 : Maintenabilité et qualité (3 semaines)	13
9.4 Phase 4 : Fonctionnalités production (4 semaines)	14
10. Conclusion	14

1. Resume Executif

Ce rapport presente l'audit technique complet de l'application KFM Delice, une plateforme de commande de repas en ligne destinee au marche guineen. L'audit couvre sept dimensions critiques : l'architecture globale, la securite, l'experience utilisateur, les fonctionnalites, la qualite du code, la performance et la maintenabilite. Chaque dimension est evaluee sur une echelle de 10 points, avec une analyse detaillee des forces, des faiblesses et des recommandations actionnables.

Categorie	Score	Appreciation
Architecture Globale	7/10	Solide mais monolithique
Securite	5/10	Insuffisant pour la production
Experience Utilisateur	8/10	Tres bonne, moderne et fluide
Fonctionnalites	8/10	Riche et bien couverte
Qualite du Code	6/10	Acceptable mais manque de rigueur
Performance	6/10	Polling excessif, pas de WebSocket
Maintenabilite	5/10	Composants monolithiques, pas de tests

Tableau 1 : Synthese des scores par categorie

SCORE GLOBAL : 6.4 / 10 - Prototype avance, pas encore production-ready. L'application presente une base fonctionnelle solide avec une experience utilisateur remarquable, mais des lacunes significatives en securite et maintenabilite bloquent le passage en production.

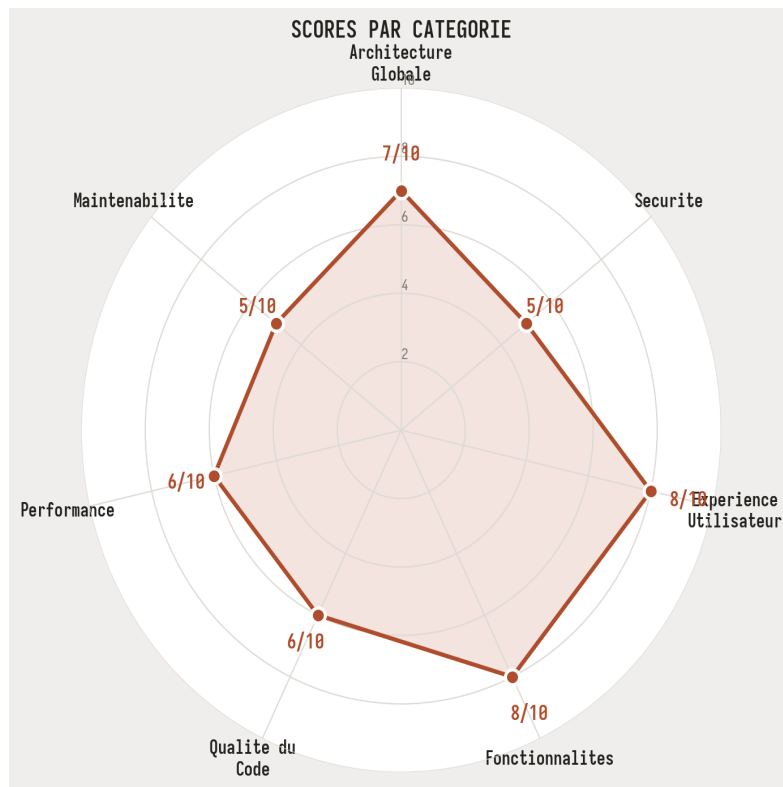


Figure 1 : Visualisation radar des scores par categorie

2. Architecture Globale - 7/10

L'architecture de KFM Delice repose sur Next.js 16 avec App Router, Prisma ORM et SQLite. L'ensemble de l'application est organisee en un depot monolithique comprenant les pages, les composants, les routes API et la logique metier. Cette approche facilite le developpement rapide et le deploiement simple, mais elle introduit des limites structurelles qui deviendront problematiques a mesure que l'application evolue et que le volume de donnees augmente.

2.1 Structure du projet

Le projet suit la convention Next.js App Router avec les repertoires standards : `src/app/` pour les pages et routes API, `src/components/` pour les composants React organises par role (admin, customer, driver, ui), et `src/lib/` pour la logique partagee. L'application compte 26 routes API, 68 gestionnaires de methodes HTTP, et des dashboards distincts pour les trois types d'utilisateurs (admin, client, livreur). La base de donnees SQLite contient 13 modeles Prisma avec des relations hierarchiques centrees sur l'entite Restaurant.

2.2 Points forts

POINT FORT : Separation claire des roles utilisateur avec des dashboards dedies (admin 15 onglets, client 7 onglets, livreur 4 onglets). L'architecture des routes API est coherente avec des endpoints RESTful bien structures et une couverture CRUD complete pour chaque entite.

POINT FORT : Utilisation moderne de Next.js App Router avec server components et streaming. Les 26 routes API couvrent l'ensemble du cycle de vie metier avec une authentification JWT et un controle d'accès basé sur les rôles (admin, manager, staff, customer, driver).

2.3 Faiblesses identifiées

Absence de couche service : Les routes API interrogent directement Prisma sans couche de service intermédiaire. Cela signifie que la logique métier (calcul de prix, gestion des statuts, règles de validation) est dispersée dans les handlers API plutôt que centralisée. Toute modification de règle métier nécessite de retrouver et modifier chaque endpoint concerné, avec un risque élevé de régression et d'incohérence.

Monolithe unique : L'ensemble de l'application (front-end, back-end, base de données) est contenu dans un seul dépôt et un seul processus. Si l'API de paiement ou le service de notification rencontre un problème, l'intégralité de l'application est impactée. Il n'existe aucune possibilité de mise à l'échelle indépendante des différents modules.

Hypothèse mono-restaurant : Le schéma Prisma et le code utilisent systématiquement `db.restaurant.findFirst()` pour récupérer le restaurant, ce qui suppose l'existence d'un seul établissement. Cette conception rend impossible le support multi-enseignes sans refonte majeure du schéma et de la logique applicative.

Base de données SQLite : SQLite ne supporte pas les écritures concurrentes, la recherche en texte intégral native, ni les requêtes complexes sur de grands volumes. En production, avec des commandes simultanées et des requêtes analytiques, SQLite deviendra un goulot d'étranglement incontournable. La migration vers PostgreSQL est indispensable.

3. Sécurité - 5/10

La sécurité est le point le plus critique de cet audit. Malgré des fondations solides (JWT avec `jsonwebtoken`, `bcrypt` pour les mots de passe, headers de sécurité en middleware), plusieurs vulnérabilités majeures rendent l'application impropre à un déploiement en production. Certaines de ces failles pourraient être exploitées pour modifier des paiements, accéder à des données sensibles ou compromettre l'intégrité du système.

3.1 Vulnérabilité critique : contournement du paiement

CRITIQUE : La route `/api/payment` accepte un paramètre `webhook=true` qui contourne complètement l'authentification. N'importe quel utilisateur peut envoyer une requête PATCH avec `{ webhook: true, id: '...', status: 'paid' }` pour marquer n'importe quel paiement comme payé sans authentification. Cette faille permet à un client malveillant de valider ses propres commandes sans payer.

Le code source dans `src/app/api/payment/route.ts` (lignes 352-353) vérifie si le paramètre `webhook` est présent et, le cas échéant, saute entièrement la vérification d'authentification admin. La

solution correcte est d'utiliser une signature HMAC ou un secret partage entre le webhook du fournisseur de paiement et l'API, plutot qu'un simple drapeau dans le corps de la requete.

3.2 Vulnerabilites de niveau moyen

ATTENTION : Modification du mot de passe admin sans verification de l'ancien mot de passe. Dans `src/app/api/admins/route.ts` (lignes 90-98), un administrateur peut changer son mot de passe sans fournir le mot de passe actuel. Contrairement aux clients qui doivent fournir `currentPassword`, les administrateurs n'ont aucune verification. Si un attaquant obtient un token JWT admin, il peut verrouiller definitivement l'acces de l'administrateur legitime.

ATTENTION : Endpoint de seed accessible sans authentication lorsqu'aucun admin n'existe. Sur un deploiement frais, un attaquant pourrait creer un administrateur avant le proprietaire legitime et prendre le controle total de l'application.

ATTENTION : Rate limiting en memoire uniquement. Les compteurs de limitation de debit sont stockes en memoire et reinitialises a chaque redemarrage du serveur. Dans un deploiement multi-instance, chaque instance maintient ses propres compteurs, ce qui multiplie le debit autorise par le nombre d'instances. La migration vers Redis ou un store partage est necessaire.

3.3 Points forts de securite

POINT FORT : Verification JWT via jose (compatible Edge Runtime) dans le middleware avec injection des informations utilisateur via les headers de requete (`x-user-id`, `x-user-type`, `x-user-role`). Les headers de securite sont appliques sur toutes les reponses : `X-Content-Type-Options`, `X-Frame-Options: DENY`, `X-XSS-Protection`, `Referrer-Policy`, `Permissions-Policy` et CSP en production.

POINT FORT : Validation Zod exhaustive : plus de 30 schemas couvrent l'ensemble des entrees (cote client et serveur). Verification des prix cote serveur pour les commandes, empechant la manipulation client-side des tarifs. Hachage bcrypt des mots de passe avec rate limiting sur les routes d'authentification (10 req/min).

4. Experience Utilisateur - 8/10

L'experience utilisateur est l'un des points forts de l'application. L'interface est moderne, fluide et bien pensee, avec une navigation intuitive, un design responsive et des retours visuels appropriés. L'utilisation de `shadcn/ui` et de Tailwind CSS confere a l'application un aspect professionnel et coherent a travers toutes les pages. Les transitions sont douces, les formulaires sont bien structures et les etats de chargement sont generalement bien geres.

4.1 Points forts UX

POINT FORT : Design responsive complet avec breakpoints Tailwind sur 30+ composants. Le dashboard admin s'adapte parfaitement du mobile au desktop avec des grilles dynamiques. Le mode

sombre est implemente nativement avec next-themes, detection des preferences systeme et toggle sur chaque dashboard.

POINT FORT : Systeme de notification multi-couches : toast Sonner avec 30+ helpers types, WebSocket temps reel avec heartbeat et reconnexion exponentielle, notifications push VAPID via service worker, et polling de fallback toutes les 30 secondes pour les statistiques admin.

POINT FORT : Etats vides bien geres : 19+ composants affichent des messages contextuels lorsqu'aucune donnee n'est disponible (Aucun client trouve, Aucun livreur, Aucun avis, etc.). Le cycle de vie complet des commandes est visualisable avec une timeline de statut claire.

4.2 Lacunes identifiees

Absence de pages d'erreur : Aucun fichier `error.tsx` n'existe dans l'application. Les erreurs runtime affichent la page d'erreur par default de Next.js, qui est en anglais et sans mise en page coherente avec le reste de l'application. De meme, aucun `not-found.tsx` n'est defini pour les erreurs 404. Cette lacune degrade significativement l'experience en cas de probleme technique.

Gestion des erreurs API silencieuse : Le hook `useAdminData` effectue 13 appels API en parallele au montage du composant. Si ces appels echouent, les erreurs sont logguees en console mais aucun message utilisateur n'est affiche. L'utilisateur se retrouve face a un spinner infini sans comprehension du probleme ni possibilite d'action (reessayer, recharger, contacter le support).

Accessibilite partielle : Les composants custom manquent d'attributs ARIA (le bouton WhatsApp n'a pas de `aria-label`, les toggles de statut driver n'ont pas de `aria-pressed`, les elements de sidebar n'ont pas de `aria-current`). Il n'existe pas de lien de navigation `skip-to-content` pour les utilisateurs de lecteurs d'ecran, et la gestion du focus dans les modales est absente.

Absence d'internationalisation : L'interface est entierement en francais sans possibilite de changement de langue. Aucune bibliotheque i18n, aucun fichier de traduction, aucun mecanisme de changement de locale n'est implemente. Pour un marche en expansion en Afrique de l'Ouest, le support multilingue (anglais, langues locales) serait un avantage competitif.

5. Fonctionnalites - 8/10

KFM Delice offre une couverture fonctionnelle remarquable pour une application a ce stade de developpement. Le cycle de vie complet des commandes est implemente, de la creation par le client jusqu'a la livraison par le driver, en passant par la preparation en cuisine et le paiement. Les 26 routes API et 68 gestionnaires HTTP couvrent l'ensemble des operations CRUD necessaires pour chaque entite metier.

5.1 Cycle de vie des commandes

Etape	Statut	Acteur	Implementa tion
Creation	pending	Client / Public	Complete
Confirmation	confirmed	Admin / Manager	Complete
Preparation	preparing	Admin / Manager	Complete
Pret	ready	Admin / Manager	Complete
Prise en charge	picking_up	Driver	Complete
En livraison	delivering	Driver	Complete
Livre	delivered	Driver	Complete
Paieement	paid	Client	Simule
Annulation	cancelled	Admin	Complete

Tableau 2 : Cycle de vie complet des commandes

5.2 Fonctionnalites implementees

Le dashboard administrateur comprend 15 onglets couvrant l'ensemble de la gestion : vue d'ensemble avec KPIs et graphiques, gestion des reservations, commandes, menu (avec upload d'images et optimisation sharp), livraisons avec suivi en temps reel, livreurs, avis clients, personnel, clients, administrateurs, factures (avec calcul de taxes), devis (avec workflow de statut et remises), depenses, paiements (avec filtre par methode), et point de vente (POS) complet avec panier et checkout. Le dashboard client offre 7 onglets et le dashboard livreur 4 onglets, chacun avec des fonctionnalites dediees.

5.3 Fonctionnalites manquantes

Fonctionnalite	Priorite	Impact
Integration Orange Money / MTN Money reelle	Critique	Paieements actuellement simules avec Math.random()
Export PDF des factures et CSV des rapports	Haute	Impossible de generer des documents comptables
Annulation de commande par le client	Haute	Seul l'admin peut annuler, frustration client
Parametres restaurant (UI d'edition)	Moyenne	Informations modifiables uniquement via seed/DB
Redemption des points de fidelite	Moyenne	Programme de fidelite affichage uniquement

Carnet d'adresses de livraison	Moyenne	Saisie manuelle a chaque commande
Journal d'audit (AuditLog)	Haute	Aucune trace de qui a modifie quoi et quand
Carte interactive (Leaflet/Mapbox)	Basse	DriverMapTab existe sans rendu de carte reel
Chat admin-client-livreur	Basse	Communication uniquement par WhatsApp externe

Tableau 3 : Fonctionnalités manquantes identifiées

5.4 Paiements simulés

ATTENTION : Les intégrations Orange Money et MTN Money sont entièrement simulées. Le code utilise `Math.random()` pour déterminer le succès ou l'échec des paiements (95% de taux de succès simulé), avec un délai artificiel de 2 secondes. Le flux OTP retourne `otpRequired: true` sans vérification réelle. La mise en production requiert le remplacement de ces simulations par les véritables API des opérateurs mobile money.

6. Qualité du Code - 6/10

La qualité du code est acceptable pour un prototype avancé mais manque de rigueur sur plusieurs aspects critiques. L'utilisation de TypeScript est systématique mais la discipline de typage est insuffisante, avec de nombreuses occurrences de types permissifs et de conversions implicites. La gestion des erreurs est inégale, oscillant entre des patterns robustes (Zod, try/catch dans les API routes) et des silences dangereux (erreurs avalées dans les hooks sans feedback utilisateur).

6.1 Typage TypeScript

Le fichier `tsconfig.json` ne active pas les options de vérification les plus strictes. L'utilisation du type `any` est répandue dans les composants et les hooks, notamment dans les réponses API et les gestionnaires d'événements. Cette permissivité masque des erreurs potentielles à la compilation et rend le refactoring risqué. L'activation progressive de `strict: true` dans `tsconfig.json`, accompagnée de la correction des erreurs résultantes, serait un premier pas essentiel.

6.2 Validation des données

POINT FORT : La validation Zod est exhaustive et cohérente : 30+ schémas couvrent toutes les entrées, avec validation à la fois côté client et côté serveur. Chaque route API utilise `safeParse()` avant tout traitement, et les schémas PATCH sont définis séparément des schémas de création pour permettre les mises à jour partielles. C'est l'un des points les plus solides du codebase.

6.3 Gestion des erreurs

La gestion des erreurs presente un contraste marque entre les API routes et les composants frontend. Cote serveur, chaque route API encapsule sa logique dans un bloc try/catch avec des reponses d'erreur structurees en francais. Cote client, en revanche, le hook `useAdminData` capture les erreurs via `console.error` sans les exposer a l'utilisateur. Si les 13 appels API paralleles echouent simultanement, l'utilisateur est confronte a un spinner permanent sans aucune indication du probleme ni moyen de relancer les requetes.

6.4 Duplication de code

Des patterns de code similaires se repetent a travers les composants d'onglets admin : la structure de tableau avec pagination, les modales de creation/edition, les filtres de recherche, et les appels API CRUD suivent tous le meme modele mais sont reimplémentes a chaque fois. L'extraction de composants generiques (`DataTable`, `CrudModal`, `SearchFilter`) reduirait significativement la duplication et faciliterait l'ajout de nouvelles entites. On estime qu'environ 40% du code des onglets admin pourrait etre factorise dans des composants partages.

7. Performance - 6/10

La performance est un domaine a ameliorer pour atteindre le niveau production. L'application presente plusieurs anti-patterns qui degradent les temps de reponse et la consommation de ressources, notamment le chargement massif de donnees au montage des composants et le recours au polling plutot qu'aux mises a jour temps reel via WebSocket. Malgre l'existence d'une infrastructure WebSocket fonctionnelle, elle n'est pas utilisee de maniere optimale.

7.1 Chargement massif de donnees

CRITIQUE : Le hook `useAdminData` charge 1000 items par entite pour 13 entites differentes au montage du dashboard admin, soit potentiellement 13 000 enregistrements charges simultanement. Ce pattern provoque des temps de chargement initiaux excessifs et une consommation memoire injustifiee. La pagination cote serveur avec des tailles de page raisonnables (20-50 items) est indispensable.

7.2 Polling vs WebSocket

L'application dispose d'une infrastructure WebSocket complete (serveur sur port 3001, hook `useWebSocket` avec reconnexion exponentielle et heartbeat), mais le dashboard admin continue d'utiliser un polling HTTP toutes les 30 secondes pour les statistiques. Cette approche genere un trafic reseau inutile et des latences de mise a jour allant jusqu'a 30 secondes. La migration vers une architecture pilotee par evenements via WebSocket pour les mises a jour temps reel (nouvelles commandes, changements de statut, positions des livreurs) est essentielle.

7.3 Absence de pagination cote serveur

Les routes API de liste (/api/orders, /api/customers, etc.) retournent l'integralite des enregistrements sans pagination. Le parametre take: 1000 est utilise comme limite artificielle dans les requetes Prisma, mais il ne s'agit pas d'une veritable pagination. L'absence de parametres skip, cursor et orderBy cote serveur oblige le client a charger et filtrer de grands volumes de donnees inutilement.

7.4 Optimisations absentes

Optimisation	Statut actuel	Impact attendu
Pagination cote serveur	Absente (take: 1000)	Reduction de 95% du volume de donnees transferees
Mise en cache (SWR/React Query)	Absente	Elimination des rechargements inutiles
Lazy loading des onglets admin	Tous charges au montage	Temps de chargement initial divise par 3-5
React.memo / useMemo	Rarement utilise	Reduction des re-rendus inutiles
Server Components (RSC)	Usage minimal	Reduction du bundle JavaScript client
Compression des images	Sharp implemente	Deja en place

Tableau 4 : Optimisations de performance identifiees

8. Maintainabilite - 5/10

La maintainabilite est le point faible de l'application, a egalite avec la securite. L'absence de tests automatises, la taille excessive de certains composants, l'absence de documentation technique et le manque de separation des responsabilites rendent les evolutions et les corrections de bugs de plus en plus couteuses et risquees au fil du temps. Ce score de 5/10 signifie que chaque modification du code a un risque significatif de regression non detectee.

8.1 Absence de tests automatises

CRITIQUE : Malgre l'existence de 12 fichiers de test dans src/__tests__/, la couverture de test est largement insuffisante. Les tests existants couvrent principalement la validation, l'authentification, la pagination et le rate limiting, mais aucune route API complete n'est testee de bout en bout. Il n'y a aucun test d'integration, aucun test E2E, et aucun test de composant React. Le ratio code de test / code applicatif est estime a moins de 5%.

Le framework de test Vitest est configure, ce qui est positif, mais l'absence de tests pour les composants React (pas de React Testing Library) et pour les flux critiques (cycle de vie d'une commande, processus de paiement, gestion des sessions) signifie que toute modification du code peut introduire des regressions sans qu'aucun test ne les detecte. La mise en place d'une pipeline CI/CD avec des seuils de

couverture minimum (80% pour les routes API, 60% pour les composants) est indispensable.

8.2 Composants monolithiques

Plusieurs composants dépassent largement la taille recommandée de 300 lignes. Le composant `AdminDashboard.tsx` agit comme un routeur d'onglets qui importe et orchestre 15 sous-composants. Certains onglets (`OrdersTab`, `MenuTab`) contiennent à la fois la logique de récupération de données, la logique de présentation et les modales d'édition dans un seul fichier. La séparation entre composants de présentation (UI pure) et composants conteneurs (logique de données) n'est pas respectée, ce qui rend les composants difficiles à tester, réutiliser et maintenir.

8.3 Relations en chaîne de caractères

ATTENTION : Les commandes, réservations et avis sont liés aux clients par `customerName` (chaîne de caractères) et non par `customerId` (cle étrangère). Cette conception viole les principes fondamentaux des bases de données relationnelles et crée des risques d'intégrité : deux clients homonymes provoquent des conflits, un changement de nom d'un client rompt ses liens avec ses commandes, et les requêtes par client nécessitent des recherches textuelles coûteuses et imprécises.

8.4 Absence de journal d'audit

Il n'existe aucun mécanisme de journalisation des actions administratives. Aucune trace n'est conservée de qui a modifié quelles données et quand. En cas de problème (données corrompues, modification abusive, erreur de manipulation), il est impossible de déterminer la cause ou de restaurer l'état antérieur. Le schéma Prisma ne contient pas de modèle `AuditLog`, et aucune logique de traçabilité n'est implémentée dans les routes API. Pour une application de restauration gérant des paiements et des données clients, cette lacune est un risque juridique et opérationnel majeur.

9. Feuille de Route vers la Production

Le passage de prototype avancé à application production-ready nécessite un effort structuré sur plusieurs phases. Les recommandations ci-dessous sont classées par priorité et estimées en semaines-homme de travail. L'ordre proposé respecte les dépendances entre les tâches et maximise l'impact de chaque phase sur la qualité globale du produit.

9.1 Phase 1 : Sécurité critique (2 semaines)

Tâche	Description	Effort
Corriger le contournement webhook	Remplacer le flag webhook par une signature HMAC avec secret partagé	2 jours

Verification mot de passe admin	Ajouter currentPassword obligatoire pour les modifications admin	0.5 jour
Protection de l'endpoint seed	Desactiver en production ou proteger par un token d'initialisation	0.5 jour
Rate limiting persistant	Migrer les compteurs vers Redis ou Upstash	1 jour
Migration vers PostgreSQL	Remplacer SQLite par PostgreSQL avec les memes schemas Prisma	3 jours
Correction des relations client	Remplacer customerName par customerId (cle etrangere)	3 jours

Tableau 5 : Phase 1 - Securite critique

9.2 Phase 2 : Performance et stabilite (2 semaines)

Tache	Description	Effort
Pagination cote serveur	Implementer skip/take/cursor sur toutes les routes de liste	3 jours
Migration WebSocket	Remplacer le polling admin par les evenements WebSocket	2 jours
Lazy loading des onglets	Charger chaque onglet admin uniquement quand il est actif	1 jour
Pages d'erreur custom	Ajouter error.tsx et not-found.tsx pour chaque section	1 jour
Gestion erreurs API frontend	Afficher des messages d'erreur et boutons retry	2 jours
SWR / React Query	Remplacer les fetch manuels par un cache intelligent	3 jours

Tableau 6 : Phase 2 - Performance et stabilite

9.3 Phase 3 : Maintenabilite et qualite (3 semaines)

Tache	Description	Effort
Tests d'integration API	Couverture 80% minimum sur les routes critiques	5 jours
Tests de composants React	React Testing Library sur les composants principaux	3 jours
Refactoring composants admin	Extraire DataTable, CrudModal, SearchFilter generiques	3 jours
TypeScript strict	Activer strict: true et corriger les erreurs	2 jours
Journal d'audit	Modele AuditLog + middleware de traabilite	2 jours

CI/CD avec seuils de couverture	Pipeline GitHub Actions avec tests et deployment	2 jours
---------------------------------	--	---------

Tableau 7 : Phase 3 - Maintenabilite et qualite

9.4 Phase 4 : Fonctionnalites production (4 semaines)

Tache	Description	Effort
Integration Orange Money reelle	API Orange Money CI avec gestion OTP et callback	5 jours
Integration MTN Money reelle	API MTN MoMo avec token et notification	5 jours
Export PDF factures	Generation PDF des factures avec ReportLab	2 jours
Export CSV rapports	Export des donnees admin en CSV	1 jour
Parametres restaurant UI	Interface d'edition des infos restaurant	1 jour
Annulation commande client	Permettre l'annulation dans un delai configurable	1 jour
Accessibilite (ARIA)	Ajouter aria-labels, skip-nav, focus management	2 jours
Persistence PushSubscription	Stocker les abonnements push en base	1 jour

Tableau 8 : Phase 4 - Fonctionnalites production

10. Conclusion

KFM Delice est un prototype avance qui demontre une comprehension solide des besoins metier du marche guineen de la livraison de repas. L'experience utilisateur est remarquable (8/10), avec une interface moderne, responsive et bien pensee. La couverture fonctionnelle est riche (8/10), avec un cycle de vie complet des commandes et des dashboards complets pour chaque role. L'architecture (7/10) est coherente mais monolithique, et les bases technologiques (Next.js, Prisma, TypeScript) sont modernes et appropriees.

Cependant, le passage en production est bloque par deux domaines critiques : la securite (5/10) et la maintenabilite (5/10). La vulnerabilite de contournement du paiement, les paiements simules, l'absence de tests automatises et les relations basees sur des chaines de caracteres plutot que des cles etrangeres sont des bloquants absolus. La performance (6/10) et la qualite du code (6/10) necessitent egalement des ameliorations significatives pour supporter un trafic reel.

La feuille de route proposee en quatre phases (securite, performance, maintenabilite, fonctionnalites) represente environ 11 semaines de travail. L'execution sequentielle de ces phases permettrait de

transformer KFM Delice d'un prototype avance en une application production-ready, capable de gerer des transactions financieres reelles et de servir des utilisateurs avec fiabilite et securite.

Dimension	Score	Objectif Production	Ecart
Architecture Globale	7/10	8/10	-1
Securite	5/10	8/10	-3
Experience Utilisateur	8/10	8/10	0
Fonctionnalites	8/10	9/10	-1
Qualite du Code	6/10	8/10	-2
Performance	6/10	8/10	-2
Maintenabilite	5/10	8/10	-3

Tableau 9 : Scores actuels vs objectifs production